Flat Baseline (M0–M4): Complete End-to-End Walkthrough

> **Purpose of this document**: written for someone with **no prior knowledge of this project**. It documents the "flat baseline" from data download, cloud removal, data cleaning, feature engineering, merging, modelling, parameter selection and backtesting, all the way to results and analysis — **every step spelled out in full**.
>
> **One-sentence definition**: the flat baseline is the **core empirical benchmark layer** of this dissertation. It "flattens" three data modalities (finance, remote sensing, shipping) into one wide table, feeds it to two machine-learning models (Ridge, XGBoost), and adds modalities one layer at a time (the ablation ladder M0→M4) to test whether each new data type actually helps predict next-week oil prices.
>
> **Last updated**: 2026-07-28 (paths → `最新待整理/`; clarifies 365×213 matrix and flat vs deep GFW lags; adds main-analysis vs robustness + RQ2 bridge; main-result numbers remain 2026-07-03/05 L4_tuned)
>
> **Experiment log**: `00_admin/最新待整理/flat_baseline_log.md`
> **Variable list**: `00_admin/最新待整理/2026-07-28_扁平模型变量清单.md`
> **Progress overview**: `00_admin/最新待整理/2026-07-15_研究方案与进度总览.md`
> **Deep counterpart**: `deep_model_full_walkthrough_EN.md`
> **Code / data**: `04_code/` · `03_data/`

Contents

---

1. What problem the project solves

Prediction target: next-week Brent crude spot price P_{t+1} (USD/barrel).

Training approach: the model does not predict the price directly. It predicts the **log return** $r_{t+1} = \ln(P_{t+1}/P_t)$, then reconstructs the price as $\hat{P}_{t+1} = P_t \cdot e^{\hat{r}}$. This is because the price level is non-stationary (trending), whereas log returns are more stationary and better suited to modelling.

Core research questions:

- **RQ1**: Can satellite remote sensing and shipping data beat a purely financial model, and beat the "random walk" strong benchmark?
- **RQ2**: Is simply "flattening and concatenating" multiple data types enough, or is a smarter "modality-aware fusion" needed?

Ablation ladder (add data layer by layer, isolate each modality's incremental value):

Model	Data used	Plain meaning
M0	Nothing learned	"next price = this price"; random walk; the simplest yet very strong benchmark
M1	Finance/macro (31 cols)	prices, inventories, VIX, gold, FX, geopolitical risk, etc.
M2	M1 + remote sensing (55 cols)	satellite features: vegetation/water/built-up/bare-soil/night-lights at 11 oil sites
M3	M1 + shipping (113 cols)	shipping features: tanker flow through 6 chokepoints, AIS vessel presence
M4	M1 + M2 + M3	finance + remote sensing + shipping all together

Two algorithms per model:

- **Ridge**: L2-regularised linear regression; robust but sensitive to high-dimensional noise
- **XGBoost**: gradient-boosted trees; captures non-linearity, more robust to high-dimensional redundancy

---

2. Step 0: Raw data download

Each of the three modalities has its own sources, all stored under `03_data/raw/` (the entire `raw/` directory is git-ignored).

2.1 M1 finance/macro

Location: `03_data/raw/01_market_financial/`

Subfolder	Source	Content	Frequency	Download
`EIA/`	US Energy Information Administration	Brent/WTI daily spot (.xls), Weekly Petroleum Status Report (stocks/production/refinery)	daily/weekly	manual .xls
`FRED/`	Federal Reserve Economic Data	VIX, dollar index, 10Y Treasury, fed funds rate, industrial-materials index	daily/monthly	manual + API
`Yahoo/`	Yahoo Finance	S&P 500 (^GSPC), oil VIX (^OVX), Brent futures (BZ=F), CAD/USD (CADUSD=X), gold (GC=F)	daily	yfinance
`Other/`	Scholar-maintained	Dallas Fed Kilian global activity index (igrea), Caldara–Iacoviello Geopolitical Risk (GPR)	monthly/daily	manual

Download script: `download_m1_raw.py`. The build script `build_m1_weekly.py` reads local files offline by default; `--online` fetches FRED / yfinance with timeout retries.

2.2 M2 remote sensing

Location: `03_data/raw/02_sentinel2/Channel B/` (exported from GEE, git-ignored)

File	Sensor	Content	Since
`sentinel2_oil_sites_monthly_indices_..._11aoi.csv`	Sentinel-2 SR Harmonized	NDVI/NDWI/NDBI/BSI monthly optical indices	2017-04
`viirs_oil_sites_monthly_nightlights_..._11aoi.csv`	VIIRS DNB	Night-time lights NTL monthly	2014-01
`sentinel2_..._watermask_..._11aoi.csv` (robustness)	Sentinel-2	with water mask + MNDWI	2017-04

Download: Google Earth Engine (GEE) JavaScript export to Google Drive, then synced locally. Scripts e.g. `extract_sentinel2_monthly_indices_gee.js`, `extract_viirs_monthly_nightlights_gee.js`.

11 oil-infrastructure AOIs (5 km circular buffer):

Short name	Full name	Type	Country
Houston	Houston Ship Channel	port	USA
Rotterdam ★	Port of Rotterdam	port	Netherlands
NingboZhoushan	Ningbo-Zhoushan Port	port	China
Jamnagar	Jamnagar Refinery	refinery	India
Jurong	Singapore Jurong Island	refinery	Singapore
Ulsan	Ulsan Refinery	refinery	South Korea
Basra	Basra Oil Terminal	terminal	Iraq
Fujairah ★	Fujairah Oil Terminal	terminal	UAE
Kharg	Kharg Island Terminal	terminal	Iran
RasTanura ★	Ras Tanura Terminal	terminal	Saudi Arabia
Yanbu	Yanbu Export Terminal	terminal	Saudi Arabia

> ★ = the best literature-supported core oil sites; the NTL anomalies of these 4 form the "literature" curated arm later.

2.3 M3 shipping

Location: `03_data/raw/03_shipping/`

Source	File	Content	Frequency
IMF PortWatch	`portwatch_chokepoints_daily.csv`	6 chokepoint transits (tanker counts/capacity)	daily

Source	File	Content	Frequency
IMF PortWatch	`portwatch_ports_daily.csv`	export/import hub tanker tonnage	daily
Global Fishing Watch	`gfw_chokepoint_vessel_presence_monthly.csv`	6 chokepoint AIS vessel presence	monthly

6 chokepoints: hormuz, suez, malacca, mandeb (Bab el-Mandeb), panama, cape (Cape of Good Hope).

---

3. Step 1: Cloud removal & monthly compositing (M2 only)

Cloud removal is a remote-sensing-specific step, done entirely at the **GEE export stage** (the build script reads already-cloud-free monthly indices).

3.1 Sentinel-2 optical (NDVI/NDWI/NDBI/BSI) — three-step cloud removal

1. **Scene pre-filter**: whole-scene `CLOUDY_PIXEL_PERCENTAGE ≤ 60` (`CLOUD_FILTER`), dropping heavily clouded images first.
2. **Double cloud mask (per pixel)**:
   - s2cloudless cloud probability `< 40` (`CLD_PRB_THRESH`) = clear sky
   - plus SCL scene-classification mask, removing SCL ∈ {3 cloud shadow, 8 medium-prob cloud, 9 high-prob cloud, 10 cirrus, 11 snow/ice}
3. **Monthly median composite**: take the **median** of all clear-sky pixels in the month (median is most robust to residual cloud) → `reduceRegions(mean)` within each 5 km AOI, `scale=20m`.

Quality records (not modelled, diagnostics only): `valid_obs_count` (number of valid scenes in the composite), `cloud_probability`. These are "how cloudy" diagnostics and are **not used as predictors** (to avoid learning "cloudiness" as a price signal).

3.2 VIIRS night-lights (NTL)

1. **Quality mask**: remove `avg_rad < 0` (stray light / artefacts).
2. **Zonal stats**: `reduceRegions(mean/max/stdDev)` within the 5 km AOI, `scale=500m`, taking `avg_rad` mean as the light level.

VIIRS has good continuity (since 2014) with almost no cloud gaps; Sentinel-2 is cloud-affected and has month-gaps at some sites early on.

3.3 Bands & formulas

Index	Formula (GEE bands)	Meaning
NDVI	(B8−B4)/(B8+B4)	vegetation greenness
NDWI	(B3−B8)/(B3+B8)	surface water/moisture
NDBI	(B11−B8)/(B11+B8)	built-up / impervious
BSI	((B11+B4)−(B8+B2))/((B11+B4)+(B8+B2))	bare soil / storage yards
NTL	VIIRS DNB `avg_rad` mean	night-time light / activity intensity

3.4 B4 water-mask robustness variant (optional)

For offshore terminals such as Basra/Kharg where NDWI is dominated by water, GEE is re-run with a water mask: per image compute MNDWI=(B3−B11)/(B3+B11), classify `MNDWI>0` as water, and mask those pixels out of NDVI/NDBI/BSI (land only). Outputs `s2_land_px` (land-pixel fraction).

---

4. Step 2: Data cleaning & feature engineering

Each modality has one build script turning raw data into a **leakage-safe weekly table**, all aligned to **week-ending Friday (W-FRI)**.

4.1 M1 finance: `build_m1_weekly.py`

Output: `m1_weekly_features.csv` (1043 weeks × 35 cols, 2006–2025; trimmed to 2019–2025 for modelling)

Pipeline:

1. Build the 2006–2025 Friday grid
2. **Daily→weekly**: `resample("W-FRI").last()` (Friday close)
3. **EIA weekly report**: align to report Friday → shift `+1 week` (report published the following Wednesday; anti-leakage)
4. **Monthly macro**: month-end align + forward-fill to weekly → shift `+1 or +5 weeks`
5. **Derived**: log returns, spreads, inventory changes, roll-week dummy, etc.

Publication lags (anti-leakage, crucial):

Source	Weekly conversion	Lag
Daily prices/macro	Friday last value	0 (available at Friday close)
EIA weekly (WPSR)	report-Friday align	**+1 week**
GPR geopolitical risk (monthly)	month-end + ffill	+1 week
Kilian REA / IMF industrial materials (monthly)	month-end + ffill	+5 weeks

31 modelled columns (after 2026-07 trimming):

Group	Variables
Prices/derived (5)	brent_price, wti_price, brent_log_return, wti_log_return, brent_wti_spread
EIA weekly (12, +1w)	crude_stocks_excl_spr, cushing_stocks, crude_production, crude_imports, crude_exports, refinery_crude_input, refinery_utilisation, gasoline_supplied, distillate_supplied, jet_fuel_supplied, crude_stocks_change, cushing_stocks_change
Macro-financial (5)	vix, dollar_index, treasury_10y, fed_funds_rate, sp500_log_return
Derived market/macro (9)	ovx, gpr, gold_return, global_econ_activity, nonoil_industrial_commodity, brent_f1_spot_log_basis, brent_roll_week, cadusd_log_return, dgs10_change

4.2 M2 remote sensing: `build_m2_weekly.py`

Output: `m2_weekly_features.csv` (365 weeks × 155 cols, 2019–2025)

The core cleaning step = converting raw levels into "within-site de-seasonalised standardised anomalies (anom)" (the main modelled form for M2). Two steps, all expanding-window and past-only (no leakage):

1. **De-seasonalise**: for each (site, index), take the historical same-month climatology `clim` (e.g. the average of all Januaries), residual `resid = level − clim`. Positive = greener/brighter than the same month in past years.
2. **Within-site z-score**: `anom = (resid − μ)/σ`, where μ, σ use `expanding(min_periods=12)` (at least 12 months of history). Gives "how many standard deviations off".

> Because ≥12 months of history are required, **the first 12 months of each series are NaN**; combined with early cloud gaps, some sites (Jurong/Ningbo/Ulsan) have more early missingness.

Three forms:

Form	Meaning	Modelled?
`level`	raw monthly value	❌ not in main analysis (scale-incomparable, seasonal)
`anom`	within-site de-seasonalised z-score	✅ **main modelled form (55 cols)**
`mom`	month-over-month diff	❌ EDA table only

Month→week alignment (as-of join, no fake ffill):

Parameter	Value	Meaning
`OBS_DAY=15`	15	representative obs date = month start + 14 days
`PUB_LAG_DAYS=15`	15	conservative availability = month end + 15 days (simulated release lag)
Alignment	`merge_asof(backward)`	each Friday takes the most recent "already-published" monthly obs
`MAX_AGE_DAYS=100`	100	older than 100 days = modality unavailable

Each week also carries `age` (staleness in days) and `mask` (availability), making missing/stale explicit rather than copying a month's value into several identical fake weekly values.

55 modelled columns: 5 indices × 11 AOIs, named `{index}_anom_{site}` (e.g. `NTL_anom_Fujairah`). **Not modelled**: 55 levels, 22 age, 22 avail (timeliness-not-signal or near-zero variance in-window).

4.3 M3 shipping: `aggregate_shipping_to_weekly.py`

Output: `m3_weekly_features.csv` (750 weeks × 123 cols; trimmed to 2019–2025)

Pipeline:

1. **PortWatch daily** → `resample("W-FRI").sum()` weekly sum → shift `+1 week`
2. **GFW monthly** → month-end align + ffill to weekly → shift `+4 weeks`
3. Concatenate the three sources on a **union index** (fixes an old inner-join sample-loss bug: 727→362)
4. **Derived**: `tanker_share`, `n_tanker_wow_pct`, `avg_tanker_size`, cross-chokepoint aggregates, etc.

Publication lags (flat arm; do not mix with deep graph):

Source	Weekly conversion	Lag	Note
PortWatch (daily)	daily → Friday sum	**+1 week**	Shared by flat M3 and deep-graph chokepoint nodes
GFW (**monthly presence**)	month-end + ffill	**+4 weeks**	**Only** the flat 113-col `gfw_*` block
GFW event / voyage / O-D (deep graph)	event → W-FRI	**+2 weeks**	See deep walkthrough; **not** flat table columns
GFW SAR dark vessels	month-end + ffill	**+4 weeks**	Deep graph node features; **not** in flat main model

> If older notes say "GFW +2 weeks" in bulk, that is the deep-graph **event/voyage** stream — **not** flat GFW presence.

113 modelled columns (full tier, main model):

Source	Cols	Content
GFW	49	6 chokepoints × (total_hours, total_vessels, cargo_hours, mom_pct, bunker_hours, other_hours, other_share, mean_presence) + 1 aggregate
PortWatch chokepoints	~57	6 × (n_tanker, n_total, capacity_tanker, capacity, tanker_share, tanker_cap_share, avg_tanker_size, n_tanker_wow_pct, capacity_tanker_4w_ma) + 3 aggregates
PortWatch ports	7	export/import hub tonnage, net, asymmetry, log-ratio, etc.

> **Why 113-col full and not 38-col core?** See §12.2 — core empirically is the weakest and non-significant for XGB; SHAP shows the most valuable derived columns all lie outside core.

---

5. Step 3: Merging into one matrix

`build_feature_matrix.py` aligns the three weekly tables on Friday and concatenates them into the single modelling dataset.

Output: `03_data/processed/merge/outputs/weekly_feature_matrix.csv`

Dimensions (disk check 2026-07-28): **365 weeks × 213 cols** = `week_ending_friday` + **212 data cols** (2019-01-04 to 2025-12-26). Tables below still split the **212 data cols** (excluding the date index).

Block	Cols	Note
Date index	1	`week_ending_friday` (not a feature)
M1 finance	31	features
M2 remote sensing	55	anom only (level/age/avail not merged)
M3 shipping	113	full tier, all shipping cols
mask (modality availability)	11	`avail_*`, **not features** (near-zero variance in-window)

Block	Cols	Note
target	2	`target_price_next`, `target_log_return_next`, generated at modelling time, **not features**

Anti-leakage re-check: the merge does not re-apply lags (EIA already +1w in M1, merge sets `EIA_WPSR_LAG_WEEKS=0` to avoid +2w); each modality lags at source, merge only re-checks; all leakage self-checks pass.

Why the window is 2019–2025: the largest intersection where all three modalities exist. PortWatch only covers 2019+, so the standard comparison window starts in 2019. Longer history: `weekly_feature_matrix_full.csv` (~1067 weeks; not used in main analysis).

Variable detail: see `2026-07-28_扁平模型变量清单.md`.

---

6. Step 4: Missing values & outliers

(Figures below are measured on the actual matrix: 365 weeks × **212 data cols** ≈ 77,380 cells; the CSV also has 1 date-index column)

6.1 Missing-value overview

Metric	Value
Total NaN	**553 (0.71%)**
Infinite values	**0**
Columns with zero NaN	134 / 212

6.2 Missingness by modality

Modality	Cols	Total NaN	Share	Where
M1 finance	31	0	0%	none
M2 remote sensing	55	**480**	2.4%	early cloud gaps + 12-month history
M3 shipping	113	71	0.2%	week-over-week first week
mask	11	0	0%	none
target	2	2	—	last week has no "next price"

M2 missingness concentrates in three sites early on: Jurong (59 weeks per index), NingboZhoushan (37), Ulsan (24), because anom needs ≥12 months of history plus early Sentinel-2 cloud gaps.

6.3 How missing values are handled

At the dataset-assembly layer (`fill_features` in `data.py`):

```
X.ffill().fillna(0.0)
```

Step	Action	Reason
① `ffill()`	forward-fill with historical values	no leakage (past only)
② `fillna(0.0)`	residual NaN → 0	neutral value (for a z-score anomaly, 0 = no anomaly)

Key: residual NaN after filling only fall in the warm-up (first 104 weeks), never in the test period → all M0–M4 land on the **exact same 257 test weeks**, so RMSE differences reflect feature content only.

6.4 How outliers are handled

No explicit deletion or winsorisation — outliers are controlled indirectly via "transformation + model regularisation". Measured extremes:

Type	Range	Handling
RS anomaly z-score	−5.96 to 9.21 (43 with |z|>5)	**kept** (genuine extreme events: COVID, Red Sea disruption)
Week-over-week wow_pct	up to ±158%	**kept** (genuine flow spikes)
inf	0	intercepted at source: anomaly ±inf→NaN; avg_size→NaN when n_tanker=0

Three indirect anti-outlier mechanisms:

1. **Transformation**: remote sensing uses within-site z-score anomalies (de-seasonalised, de-scaled); finance uses log returns (not price levels)
2. **Ridge**: StandardScaler + strong L2 regularisation (α up to 1000) suppresses extremes
3. **XGBoost**: trees split on thresholds, inherently robust to outliers/scale

Keeping genuine extremes is deliberate — they correspond to COVID, Red Sea/Houthi and similar real events; deleting them would discard signal.

---

7. Step 5: Dataset assembly for modelling (`data.py`)

7.1 Column selection (by modality)

Columns are selected by the dictionary's `modality` field, never `target_*` or `avail_*` (mask).

7.2 Lag flattening (lookback=4)

Each feature is flattened over the past 4 weeks: `{var}_lag0` (this week), `_lag1`, `_lag2`, `_lag3` (3 weeks ago). Modelled dimension = raw cols × 4.

Model	Raw feature cols	×4 lag	Modelled dim
M0	0 (rule benchmark)	—	—
M1	31	×4	**124**
M2	31+55=86	×4	**344**
M3	31+113=144	×4	**576**

Model	Raw feature cols	×4 lag	Modelled dim
M4	31+55+113=199	×4	**796**

7.3 Target construction

- Training label: `r_next = ln(P.shift(-1)/P)`, indexed at t, representing r_{t+1}
- Reconstruction: $\hat{P}_{t+1} = P_t \cdot e^{\hat{r}}$
- Features **never contain** target or mask

---

8. Step 6: Models & parameter selection

8.1 M0 random walk (built-in rule, not a trained model)

```
r_hat = 0  →  P_hat = P_t   ("next price = this price")
```

Emitted every week as the benchmark. M0 RMSE ≈ **4.152**.

8.2 Ridge (linear + strong regularisation)

Pipeline (fit inside the training fold only, no leakage):

```
VarianceThreshold(0.0)  →  StandardScaler()  →  Ridge(alpha)
```

- Default `alpha=10.0`
- Tuning grid: `alpha ∈ {0.1, 1.0, 10.0, 100.0, 1000.0}`
- Logic: higher dimension needs larger α; L4 untuned RMSE 5.91, tuned drops to 4.26

8.3 XGBoost (gradient-boosted trees)

Pipeline: `VarianceThreshold(0.0) → XGBRegressor` (trees need no scaling)

Default params: `n_estimators=300, max_depth=3, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0`

Tuning grid (2×2×2=8 combos):

Param	Candidates
max_depth	2, 3
learning_rate	0.03, 0.05
n_estimators	200, 400
subsample	0.8 (fixed)
colsample_bytree	0.8 (fixed)
reg_lambda	1.0 (fixed)

Anti-overfitting design (crucial for small-sample high-dimension): shallow trees (depth 2–3), 80% row/column subsampling, L2 regularisation.

8.4 How parameters are chosen (inner validation)

Within each refit fold:

```
Training fold (e.g. 200 weeks)
  ├── earlier part → fit
  └── last 52 weeks → validate, pick the hyperparams with lowest RMSE
If the fold < 82 weeks (52+30) → use defaults
```

After selecting the best hyperparams, the model is retrained on the **full training fold**, then predicts. Tuning **never touches the test set**.

---

9. Step 7: Backtesting protocol (how training actually works)

An **expanding-window rolling-origin (walk-forward)** backtest. Code: `04_code/src/backtest/rolling.py`.

9.1 Key parameters

Parameter	Value	Meaning
Total weeks	365	2019-01-04 to 2025-12-26
lookback	4 weeks	each feature flattened over this + 3 prior weeks
min_train	104 weeks	first 104 weeks accumulate data only, no forecast (≈2-year warm-up)
retrain_every	13 weeks	refit once per quarter
val_weeks	52 weeks	last 52 weeks of the training fold for inner validation
Test weeks	257	≈2021–2025
seed	42	random seed

9.2 Core idea (analogy)

Imagine you are an analyst in early 2021, forecasting next week's oil price every Friday: you can only train on data "before today"; as time advances your history grows (expanding window); you don't retrain weekly but update the model once a quarter (every 13 weeks).

9.3 Step by step

Phase 1 (weeks 1–104, warm-up): accumulate data only, no forecasts.

Phase 2 (week 105, first forecast):

1. Training data = weeks 1–104 (104 samples)
2. Inner tuning: split the training set into "earlier fit + last 52 weeks validate"; Ridge tries 5 α's, XGB tries 8 combos, each picks the lowest validation RMSE
3. Retrain Ridge and XGB on all 104 weeks with the best params

4. Forecast next week's price for week 105
5. Record the M0 benchmark

Phase 3 (weeks 106–117): reuse the week-105 models to forecast week by week, no retrain.

Phase 4 (week 118, second refit): `(118−104)%13==0` triggers a refit; training data = weeks 1–117 (now larger); re-tune and retrain.

Loop to the end:

```
Week 105  train(104 samples) → forecast 105–117
Week 118  train(117 samples) → forecast 118–130
...
Last time train(~360 samples) → forecast final weeks
```

9.4 Precise answer to "how many models / training weeks / validation weeks"

Question	Answer
How many models trained	2 per fold (Ridge+XGB); ~20 folds × 2 ≈ 40 fits; M0 not trained
Training weeks	expanding: 104 (first) → ~360 (last)
Validation weeks	the **last 52 weeks** of each training fold (then retrain on full fold)
Test weeks	**257** (2021–2025), shared by M0–M4

9.5 Why designed this way

Design	Purpose
Train on past only	strict no leakage (walk-forward)
Expanding window	mimics reality: more history over time
min_train=104	first forecast already has 2 years + enough for 52-week validation + matches M2's 12-month anomaly history
retrain_every=13	quarterly update, balancing timeliness and compute
Same protocol M0–M4	differences come from data/model only — fair

9.6 Literature support (this is the standard oil-forecasting paradigm)

Paper	Backtest method
P053 Alquist, Kilian & Vigfusson (2013)	real-time alignment + recursive/expanding window + DM test; random walk as strong benchmark
P054 Baumeister & Kilian (2015)	real-time recursive forecast combinations
P072 Costa et al. (2021)	expanding-window pseudo-out-of-sample
P076 Yılmaz & Zehir (2026)	rolling out-of-sample + multi-horizon + DM

Paper	Backtest method
P025 Hao & Wang (2023, remote-sensing→oil)	recursive real-time OOS + Clark–West

10. Step 8: Main results

Main baseline (L4_tuned, 257 test weeks, M0 RMSE=4.152):

Model	RMSE ($/bbl)	skill vs M0	CW_p vs M1 (modality significant?)	Verdict
M0_RW	4.152	—	—	random-walk benchmark
M1_Flat_Ridge	4.256	−2.5%	—	financial baseline
M1_Flat_XGB	4.368	−5.2%	—	one of the best RMSE
M2_Flat_Ridge	4.414	−6.3%	0.474	+RS, linear not sig.
M2_Flat_XGB	4.440	−6.9%	**0.085** ❌	+RS, marginally not sig.
M3_Flat_Ridge	4.430	−6.7%	0.264	+shipping
M3_Flat_XGB	4.429	−6.7%	**0.0002** ✅	+shipping, highly sig.
M4_Flat_Ridge	4.525	−9.0%	0.314	all, linear not sig.
M4_Flat_XGB	4.507	−8.6%	**0.009** ✅	all, sig. but worst RMSE

Three core findings:

1. **No model beats M0** (all skill < 0) — weekly oil prices are very close to a random walk, an expected strong benchmark.
2. **RS alone (M2) not significant** (0.085); **shipping alone (M3) significant** (0.0002); **all modalities (M4) significant** (0.009).
3. **Counter-intuitive point**: M4 is significant yet its RMSE (4.507) is worse than M1 (4.368) — because RMSE asks "how large is the final error", while Clark-West asks "after penalising the larger model for estimating extra parameters, does the added feature carry directional signal". Signal exists, but flat concatenation cannot exploit it.

11. Step 9: How results are analysed

11.1 Three-tier evidence chain

```
Tier 1 (accuracy): RMSE / MAE / skill vs M0  → "how large is the error? beats
M0?"
Tier 2 (significance): DM / Clark-West         → "is the improvement significant
or chance?"
```

```
Tier 3 (interpretation): SHAP / LOAO / LOMO    → "which variables? which sites matter?"
```

11.2 Roles of metrics and tests

Tool	Use	Type
RMSE / MAE	average forecast error	accuracy metric
skill vs M0	1 – RMSE/RMSE_M0	relative to random walk
DirAcc	direction hit rate	accuracy metric
Diebold–Mariano	vs M0 (non-nested)	econometric test
Clark–West	vs M1 (nested)	correct test for nested models
SHAP	which variables/modalities used	ML interpretability (≠ causation, ≠ significance)

Why DM for M0 and CW for M1: M2/M3/M4 are nested extensions of M1 (M1 plus features). For nested models the standard DM is biased (the larger model estimates extra parameters whose true value is zero); Clark-West corrects exactly this bias. M0 is non-nested vs the models, so DM is used.

11.3 Interpreting "significant but worse RMSE" (M4)

- **RMSE**: final error → M4(4.507) > M1(4.368), adding features made it worse
- **Clark-West**: after removing the noise penalty, does the added modality carry directional signal → p=0.009 significant

Analogy: the added modality is like a verbose analyst — it has a little genuine signal (CW significant) but carries too much noise (RMSE worsens). → Flat concatenation cannot exploit it, motivating modality-aware fusion (RQ2).

---

12. Step 10: Robustness checks (all completed)

12.1 Overview of extended analyses

Analysis	Modality	Key finding
lookback sweep	M1	tuning matters most; Ridge worsens with lookback, XGB immune
literature arm (4 NTL cols)	M2	XGB CW=0.022 ✅, few-and-precise > all 55
leave-one-AOI-out	M2	all sites |dRMSE|≤0.05, no essential positive-contribution site
SHAP	M2	NDVI/NDWI on top; financial features still dominate Top-5
C2 dim-reduction (PCA/ElasticNet)	M2	PCA gives XGB the biggest gain (decorrelation)
B4 water mask	M2	XGB CW 0.085→0.028 ✅ (significant after removing water noise)
2×2 sparsity six arms	M2	cutting sites or indices either way crosses into significance (aoi4 strongest 0.0055)

Analysis	Modality	Key finding
leave-one-channel-out	M3	core(38) not sig.; full/tanker/portwatch sig.
SHAP	M3	Top: hormuz tanker share, suez wow_pct (Red Sea mechanism)
leave-one-modality-out	M4	M1-only best RMSE globally; adding M2 to M1+M3 worsens it
SHAP modality shares	M4	shipping 51.8% > finance 33.7% > RS 14.5%

12.2 Why shipping uses 113-col full, not 38-col core

Leave-one-channel-out results:

Shipping subset	XGB RMSE	CW_p vs M1	Sig.
core (38)	4.476	0.096	❌
full (113)	**4.429**	**0.0002**	✅
tanker-only	4.343	0.0018	✅
portwatch-only	4.356	0.0003	✅

- **core is the weakest and the only non-significant one**
- SHAP Top-10 are all derived columns dropped by core (tanker_share, n_tanker_wow_pct, tanker_cap_share, avg_tanker_size)
- Conclusion: **the hand-picked core happens to delete the shipping signal XGB values most** → switch to full
- This is opposite to M2's "few-and-precise" — an M3-specific phenomenon

---

13. Main analysis vs robustness + bridge to RQ2

13.1 Main-analysis specification (formal answers; do not cherry-pick)

Layer	Main-analysis spec	Why this one
M0	Random walk $\hat{P} = P_t$	Theoretical benchmark
M1	Finance 31 cols, **L4_tuned**	Supervisor 4-week lookback + full finance set
M2	Full 11-site **anom-55**	Most naive "stuff everything in"; no hand-picking
M3	Shipping **full 113** cols	Full shipping (hand-picked core is worse for XGB)
M4	M1+M2+M3 full	Full multimodal flat concat

Robustness arms (literature / aoi4 / watermask / LOAO / LOCHO / LOMO / lookback sweep, etc.) only answer "does the conclusion survive a different subset or denoising?" — they **reinforce** the main conclusion and are **not promoted** to main analysis (otherwise = p-hacking, and it would undercut the RQ2 story that "flat cannot harvest the signal → need fusion").

13.2 Bridge to RQ2 (deep representation-level fusion)

The flat layer's honest negative results (none beat M0; full M2 not significant; M3/M4 have CW signal but no RMSE gain) are exactly the motivation for the method-integration layer. Next:

- **Full deep walkthrough**: `deep_model_full_walkthrough_EN.md`
- **RQ2 pairing table** (matched information-set Flat XGB vs Deep): progress overview §2.6 / deep walkthrough §12.5
    - Takeaway: deep significantly beats flat XGB on M3/M4 (with shipping); main model remains **Gated**; xattn is strong on a single seed but unstable

---

14. Document index & reproduction

Use	Path
This file (flat full walkthrough)	`00_admin/最新待整理/flat_baseline_full_walkthrough_EN.md`
Flat experiment number log	`00_admin/最新待整理/flat_baseline_log.md` (§7–§12; §13 deep first run is **lb=8 history** — official deep results are in the deep walkthrough)
Variable list	`00_admin/最新待整理/2026-07-28_扁平模型变量清单.md`
Progress overview	`00_admin/最新待整理/2026-07-15_研究方案与进度总览.md`
Deep walkthrough	`00_admin/最新待整理/deep_model_full_walkthrough_EN.md`
Feature matrix	`03_data/processed/merge/outputs/weekly_feature_matrix.csv`
Flat entry	`04_code/scripts/flat/run_baseline.py` + `flat/M{1..4}_Flat/`
Outputs	`05_outputs/baselines/Flat/M*_Flat/`

14.1 Reproduction commands

```
# 1) Data build
python3 03_data/processed/M1/py/build_m1_weekly.py
python3 03_data/processed/M2/py/build_m2_weekly.py
python3 03_data/processed/M2/py/build_m2_weekly.py --watermask   # B4
python3 03_data/processed/M3/py/aggregate_shipping_to_weekly.py
python3 03_data/processed/merge/py/build_feature_matrix.py

# 2) Main baselines
python3 04_code/scripts/flat/run_baseline.py --modality M1
python3 04_code/scripts/flat/run_baseline.py --modality M2 --m2-features anom
python3 04_code/scripts/flat/run_baseline.py --modality M3
python3 04_code/scripts/flat/run_baseline.py --modality M4

# 3) Robustness
python3 04_code/scripts/flat/M1_Flat/sweep_m1.py
python3 04_code/scripts/flat/M2_Flat/shap_m2.py
python3 04_code/scripts/flat/M2_Flat/robustness_m2.py
python3 04_code/scripts/flat/M2_Flat/sweep_m2.py
python3 04_code/scripts/flat/M3_Flat/shap_m3.py
python3 04_code/scripts/flat/M3_Flat/robustness_m3.py
python3 04_code/scripts/flat/M4_Flat/shap_m4.py
python3 04_code/scripts/flat/M4_Flat/robustness_m4.py
```

14.2 Output directories

```
05_outputs/baselines/Flat/
  M1_Flat/  baseline_metrics.csv, baseline_predictions.csv, backtest.png,
sweep_*
  M2_Flat/  baseline_metrics_anom.csv, shap_*.csv, c2_*, loao_*, sweep_*,
watermask
  M3_Flat/  baseline_metrics.csv, shap_*, robustness_m3_*, sweep_*
  M4_Flat/  baseline_metrics_anom.csv, shap_*, robustness_m4_*, sweep_*
```

14.3 Code structure

```
04_code/
  src/backtest/
    data.py     load matrix, select cols, lag-flatten, fill missing, build
target
    models.py   Ridge/XGB Pipeline factories + tuning grids
    rolling.py  rolling-origin backtest loop + inner tuning
    metrics.py  RMSE/MAE/DirAcc + DM(vs M0) + Clark-West(vs M1)
  scripts/
    flat/run_baseline.py     single entry point M0–M4
    flat/M1_Flat … M4_Flat/  per-modality sweep / SHAP / robustness
  deep/                      run_deep_* + diagnostics
  tools/                     subperiod_eval, migrate, relocate
```